

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

潘玉奇

第4章 数组

◆ 4.1 一维数组

◆ 4.2 二维数组

◆ 4.3 字符数组

4.1 一维数组

4.1.1 一维数组的引出

【问题描述4.1】有一个班30名学生参加了C语言考试，要求编写程序，输入每个同学的成绩，计算和输出平均成绩，并输出所有学生的成绩。

分析：根据前面所学知识，你能完成的任务是输入每个同学的成绩，计算和输出平均成绩

这里只使用1个变量score，虽然通过循环输入了30个学生的成绩，但循环结束后score中只是第30个学生的成绩，前面29个成绩并没保存下来。

需要解决的问题：如何保存30名学生的成绩，并能方便的进行输入、输出、计算等操作。

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i;
5.     double sum, score, ave;
6.     sum=0;
7.     printf("input 30 students' scores:\n");
8.     for( i=1; i<=30; i++)
9.     { scanf("%lf", &score);
10.        sum=sum + score;
11.    }
12.    ave=sum/30;
13.    printf("ave=%lf", ave);
14.    return 0;
15. }
```

4.1.1 一维数组的引出

需要解决的问题：如何保存30名学生的成绩，并能方便的进行输入、输出、计算等操作。

- 因为现在要保存30个学生的成绩，那就不能只使用1个变量，而需要30个变量，这样一来输入、输出、计算都会变得繁琐。
- 我们希望能有一种数据类型可以保存一组数据，并可以方便地对这组数据进行输入、输出、计算等操作，由此我们引出数组类型。
- 说明一个含有30个元素的数组，每个数组元素存放一个学生的成绩，而成绩的输入、输出、计算都可以通过循环来实现。

◆ **数组是一种构造数据类型**，这种类型与基本数据类型(整型、实型、字符型)是不同的。基本类型的值是不可分解的，而构造类型的值可以分解成若干个“元素”或“成员”，每个元素都是一个基本数据类型的数据，或者是一个构造类型的数据。

4.1.1 一维数组的引出

//使用数组的代码

```
1. #include <stdio.h>
2. int main()
3. {
4.     double sum, ave, score[30];
5.     int i;
6.     sum=0;
7.     printf("input 30 students' scores:\n");
8.     for( i=0; i<30; i++)
9.     { scanf("%lf", &score[i] );
10.        sum=sum + score[i];
11.    }
12.    ave=sum/30
13.    for( i=0; i<30; i++)
14.        printf("%6.2lf", score[i] );
15.    printf("ave=%6.2lf\n", ave);
16.    return 0;
17. }
```

//未使用数组的代码

```
1. #include <stdio.h>
2. int main()
3. {
4.     int i;
5.     double sum, score, ave;
6.     sum=0;
7.     printf("input 30 students' scores:\n");
8.     for( i=1; i<=30; i++)
9.     { scanf("%lf", &score);
10.        sum=sum + score;
11.    }
12.    ave=sum/30;
13.    printf("ave=%lf", ave);
14.    return 0;
15. }
```

4.1.2 一维数组的定义和引用

1. 数组的概念: 由具有**相同类型**的**固定数量**的元素组成的集合
2. 一维数组的定义格式

类型标识符 数组名[整型常量表达式];

例: int a[5];

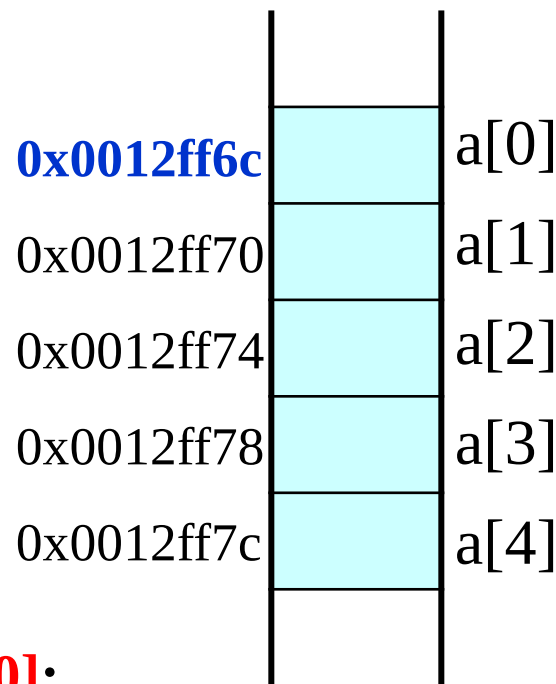
➤ 说明

(1) 类型标识符表示数组的元素具有统一的数据类型, 例中数组a的元素都是整型。

(2) 数组名是用户定义的标识符, **数组名表示数组的首地址**, 即第一个数组元素的地址。
一维数组在内存中占用一段连续的内存单元, 各元素按下标顺序依次存放。

数组a的地址就用数组名a来表示, **a等价于&a[0];**

每个数组元素的地址可表示为: **&a[0], &a[1] ... &a[4]**。



4.1.2 一维数组的定义和引用

2. 一维数组的定义格式

类型标识符 数组名[整型常量表达式];

例: int a[5];

➤ 说明

(3) 整型常量表达式定义了数组的长度，即数组中元素的个数，
例中数组a有5个元素。

注意：方括号内只能写整型常量表达式，如写成变量或其他类型的表达式都是错的。

#define SIZE 10

int a[2+3]; ✓

float b[SIZE]; ✓

double c[6.5]; ✗

int n=6;

int a[n]; ✗

int n;

scanf("%d", &n);

int a[n]; ✗

4.1.2 一维数组的定义和引用

3. 数组元素: 每一个数组元素都是一个变量, 为了与一般的变量相区别, 称数组元素为**下标变量**。

4. 数组元素的引用形式:

数组名[下标]

例: **int a[5];**

➤ 说明

(1) 数组元素的**下标由零开始**。

(2) 最后一个数组元素的下标是数组长度减一。

数组a[5]包含5个数组元素: **a[0]**, a[1], a[2], a[3], **a[4]**。

(3) 下标可以是整型常量、变量或整型表达式

如: **int a[10], i=3;**

a[1]=90; a[i]=100; a[2+i]=85;

C程序在运行中系统并不自动检验数组元素的下标是否越界。数组两端都可能因越界而破坏其它存储单元中的数据, 因此应特别注意保证数组下标不要越界。

4.1.3 一维数组的初始化

1. 数组初始化的概念

在定义一维数组时对各元素指定初始值称为数组的初始化

2. 初始化的几种方式

(1) 对数组的全体元素指定初值, 初值用 **{ }** 括起来, 数据之间用逗号分开。

例如: `int a[5] = { 1, 3, 5, 7, 9 };`

1	3	5	7	9
---	---	---	---	---

(2) 对数组中部分元素指定初值, 其余元素系统会自动赋缺省值0

例如: `int a[5] = { 1, 3, 5 };`

1	3	5	0	0
---	---	---	---	---

(3) 使数组中的全部元素初始值都为 0

例如: `int a[5] = { 0 };`

0	0	0	0	0
---	---	---	---	---

4.1.3 一维数组的初始化

➤ 提问：数组的全部元素初值都为1怎么进行初始化？

`int a[5] = { 1 }; ❌`

1	0	0	0	0
---	---	---	---	---

`int a[5] = { 1,1,1,1,1 }; ✔`

1	1	1	1	1
---	---	---	---	---

(4) 对全部元素赋初值，可以不指定数组长度，其长度由初值个数自动确定

例如： `int a[ ] = { 1, 3, 5 }; 等价于 int a[3] = { 1, 3, 5 };`

(5) 不允许数组长度小于给出初值个数

例如： `int a[3] = { 1, 2, 3, 4 };`

编译时报错：

VS2013中显示 error C2078:初始值设定项太多

Dev-C++ 5.11中显示 [Error] too many initializers for 'int [3]'

4.1.3 一维数组的初始化

- 注意：数组除定义时初值外，不能对一个数组整体赋值。

例如：

```
int i, a[5] = { 1, 2, 3, 4, 5 }, b[5];
```

```
b=a; X
```

希望将数组a 的数据全部赋值给数组b

错误原因：赋值语句b=a;是非法的，因为a和b分别代表了数组a和b的地址，都是地址常量，而赋值运算符左侧必须是变量。

- 提问：如何实现将数组a的全部元素赋值给对应的数组b？

```
int i, a[5] = { 1, 2, 3, 4, 5 }, b[5];
```

```
for( i=0; i<5; i++ )
```

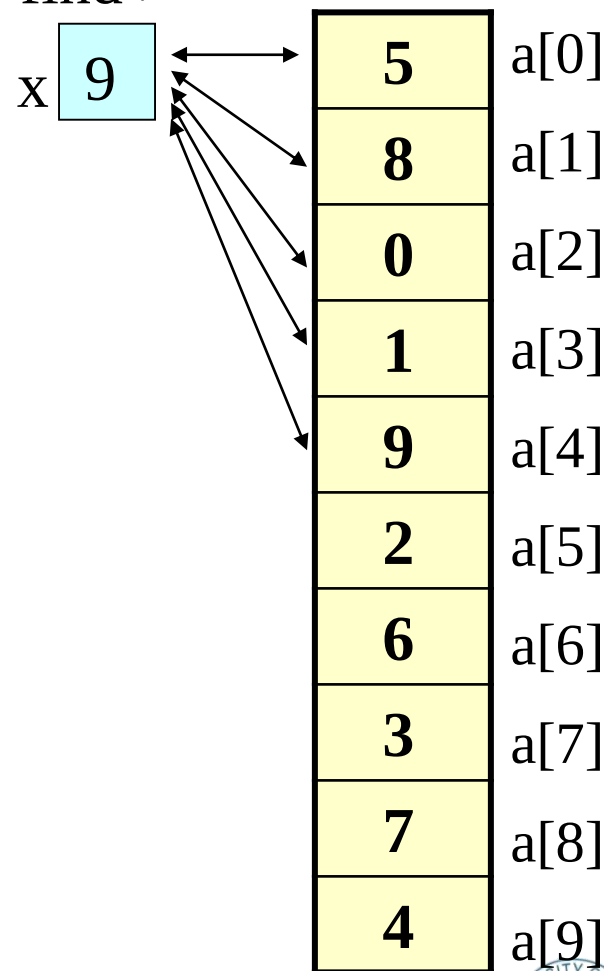
```
    b[i]=a[i];
```

// 用循环对每个元素进行赋值

4.1.4 一维数组的简单应用

【例4.1】 输入一个数据，在已知10个整数的数组中查找是否有该数据，找到输出“find!”,没找到输出“no find!”

```
#include <stdio.h>
int main( )
{ int i , x, a[10]={ 5, 8, 0, 1, 9, 2, 6, 3, 7, 4 };
  scanf("%d", &x);
  for ( i=0; i<10; i++)
  { if ( x==a[i] )
    { printf("find! \n");
      break;          //找到x后立刻结束循环
    }
  }
  if ( i==10 ) printf("no find!\n");
  return 0;
}
```



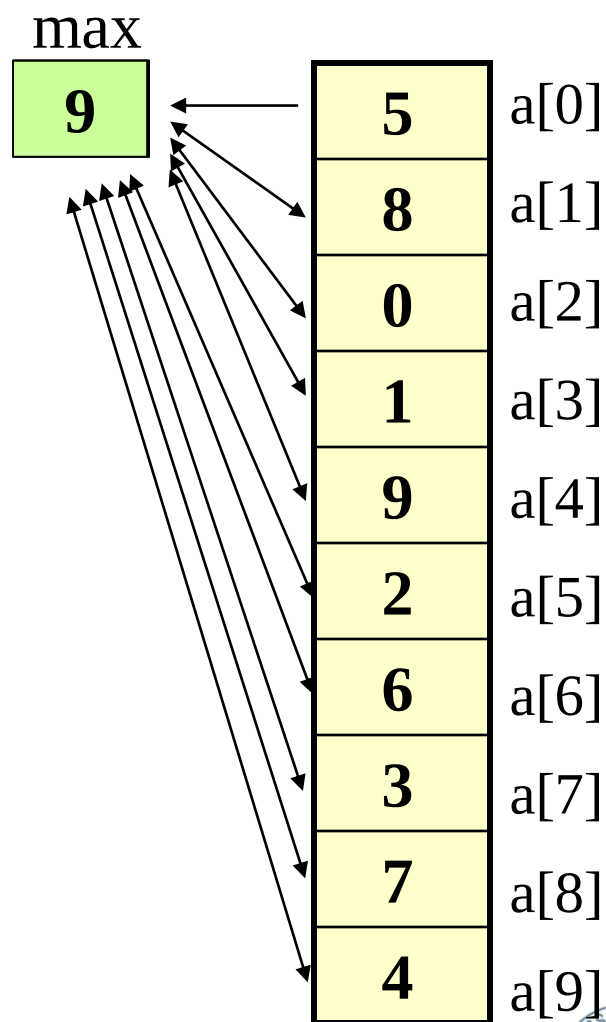
4.1.4 一维数组的简单应用

【例4.2】任意输入10个整数，找出其中的最大数输出。

分析：还记得求最大数的方法吗？

设置一个变量max，用来存放当前为止找到的最大数。先给max赋初值，把数组的第1个元素赋给max，然后用max和后面的元素依次进行比较，如果发现数据大于max，就将该数赋给max，全部数据都比较完后，max里存放的就是这一组数据中的最大数。

```
step1: max=a[0];  
step2: for( i=1; i<10; i++)  
        { if( a[i]>max )  
            max=a[i];  
        }  
step3: 输出max
```



4.1.4 一维数组的简单应用

【练习4.1】任意输入10个整数，将这10个数逆置。

提示：什么是逆置？

假设数组a有10个元素，逆置就是将a[0]和a[9]交换，a[1]和a[8]交换...，最后a[4]和a[5]交换。

```
#include<stdio.h>
```

```
int main( )
```

```
{ int i, t, a[10];
```

```
  for ( i=0; i<10; i++)
```

```
    scanf("%d", &a[i]);
```

```
  for(i=0; _____; i++)
```

```
  { 两个元素交换 }
```

```
  for( i=0; i<10; i++)
```

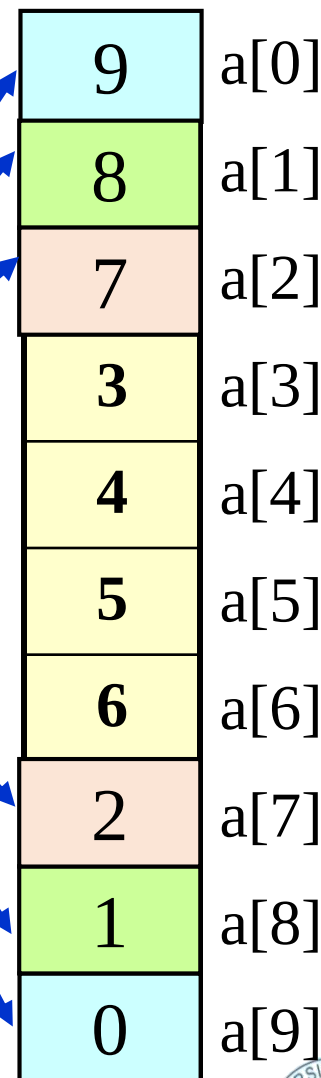
```
    printf("%3d ", a[i]);
```

```
  return 0;
```

```
}
```

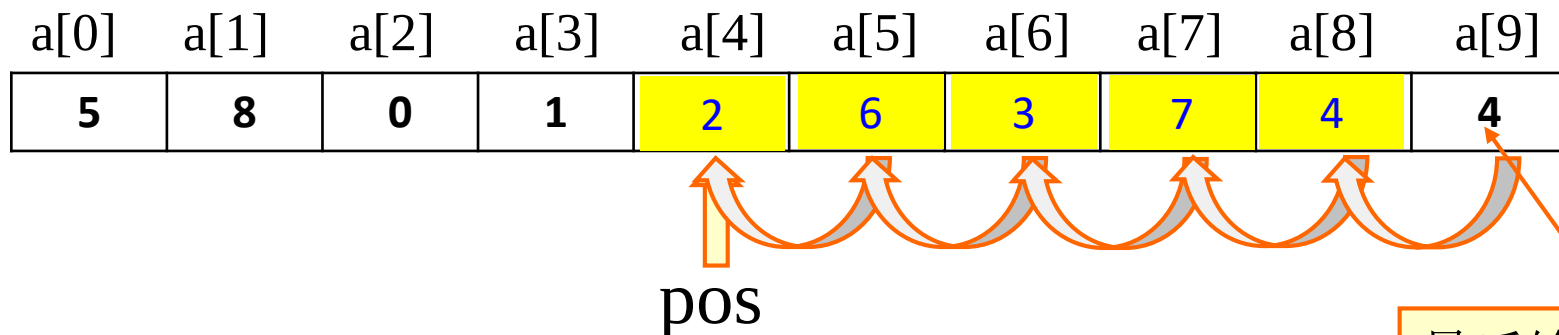
$a[0] \rightleftharpoons a[9]$

```
for(i=0; i<10/2; i++)  
{  t=a[i];  
  a[i]=a[10-1-i];  
  a[10-1-i]=t;  
}
```



4.1.4 一维数组的简单应用

【例4.3】已知一个整型数组a[10]，输入一个删除位置pos，在数组中删除元素a[pos]。假设输入pos=4，即要删除a[4]。



删除元素a[4]，那它后面的元素应该依次向前移动，即用后面的元素“覆盖”前面的元素

最后输出时
不要输出这
个元素

a[4]=a[5]; a[5]=a[6]; a[6]=a[7]; a[7]=a[8]; a[8]=a[9];

for (i=4; i<9; i++)
a[i]=a[i+1];



for (i=pos; i<9; i++)
a[i]=a[i+1];

4.1.4 一维数组的简单应用

➤ 【例4.3】在数组中删除pos位置上的元素的参考代码1

```
#include<stdio.h>
```

```
int main()
```

```
{ int a[10]={5, 8, 0, 1, 9, 2, 6, 3, 7, 4}, pos, i;
```

```
scanf("%d", &pos);
```

```
for( i=pos; i<9; i++)
```

```
    a[i]=a[i+1];
```

```
for(i=0; i<9; i++)
```

```
    printf("%d,", a[i]);
```

```
return 0;
```

```
}
```

将数组初始化改为
输入数据更好一些

// 从pos开始后面的元素依次向前移动

//注意这里是i<9, 只输出9个数

输入:

4

输出:

5,8,0,1,2,6,3,7,4,

最后的逗号
多余, 应将其去掉

输入:

12

输出:

5,8,0,1,9,2,6,3,7,

输入一个不合法的
删除位置, 没有任何
提示, 没有删除
数组元素, 但输出
时没有了元素a[9]

4.1.4 一维数组的简单应用

➤ 【例4.3】改进后的参考代码2

```
#include<stdio.h>
```

```
#define N 10           // 定义符号常量N，便于修改数组的大小
```

```
int main()
```

```
{   int a[N], pos, i;
```

```
    for(i=0; i<N; i++) scanf("%d", &a[i]);    // 数组元素由键盘输入
```

```
    scanf("%d", &pos);
```

```
    if(pos>N-1) printf("Failure!\n");
```

pos值不合法直接输出提示信息，不需要执行删除操作

```
    else
```

```
        { for(i=pos; i<N-1; i++)
```

```
            a[i]=a[i+1];
```

```
            printf("Success!\n");
```

```
            for(i=0; i<N-2; i++)
```

```
                printf("%d,", a[i]);
```

```
            printf("%d\n", a[i]);
```

```
        }
```

pos值合法时执行这部分语句，完成删除工作

//单独输出元素a[N-2]，去掉最后的逗号
// 注意最后1个元素a[N-1]是不输出的

```
    return 0;
```

```
}
```

4.1.4 一维数组的简单应用

【练习4.2】已知一个整型数组a[10]，先输入10个元素的值，然后输入一个整数x，如果数组中有x，则将x从数组中删除，并输出删除x后的数组；如果数组中没有x，则输出“Failure!”

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
5	8	0	1	9	2	6	3	7	4

假设输入x=6，删除x后数组状态如下：

5	8	0	1	9	2	3	7	4	4
---	---	---	---	---	---	---	---	---	---

假设输入x=20，则应输出“Failure!”，而数组状态不变

测试数据1：

输入：

33 15 21 39 23 41 26 18 11 37

18

输出：

33 15 21 39 23 41 26 11 37

测试数据2：

输入：

33 15 21 39 23 41 26 18 11 37

99

输出：

Failure!

4.1.4 一维数组的简单应用

【例4.4】已知一个整型数组a[10]，其中已经存放有n个整数(n是小于10的)，**且这n个数是从小到大排好序的**，现在输入一个整数x，把x插入到这个数组中，要求插入x后还是保持数据的有序性。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
4	7	9	15	18	22	26	x		

分析：

整数x的插入位置分三种情况：最前面，中间某位置，最后面

最简单的是把x插在最后面，直接赋值就行，其他不用做什么。

插在最前面和插在中间的处理方法实际上是一样的，我们需要把插入位置开始的元素全部向后移动一个位置，然后把x插入到这个空出来的位置。

4.1.4 一维数组的简单应用

➤ 把x插在最前面和插在中间的处理方法

从有数据的最后元素开始和x的值进行比较，如果 $x < \text{元素值}$ ，说明x应该放在该元素之前，把当前这个元素向后移动一个位置，然后继续用x和前一个元素比较，... 直到 $x > \text{某个元素}$ 时停止，最后把x插入到这个空出来的位置上。

假设输入 $x=20$

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
4	7	9	15	18	20	22	26		

插入x

$x < a[6]$, $a[6]$ 向后移动, 即 $a[7]=a[6]$;

$x < a[5]$, $a[5]$ 向后移动, 即 $a[6]=a[5]$;

$x > a[4]$, 说明x应该放在 $a[4]$ 的后面, 即 $a[5]=x$;

4.1.4 一维数组的简单应用

➤ 【例4.4】在有序数组中插入一个元素的参考代码

```
#include <stdio.h>
```

```
int main()
```

```
{ int a[10], n, i, x;
```

```
    printf("input n:"); scanf("%d", &n);
```

```
    for(i=0; i<n; i++) scanf("%d", &a[i]);
```

```
    printf("input x:"); scanf("%d", &x);
```

```
    for( i=n-1; i>=0 && x<a[i]; i-- )
```

```
    { a[i+1]=a[i]; } //元素后移一个位置
```

```
    a[i+1]=x; //插入x
```

```
    printf("%d", a[0]); //先单独输出a[0]
```

```
    for(i=1; i<=n; i++)
```

```
        printf(",%d", a[i]); //输出其他元素时先输出1个逗号
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

如果输入的x大于最后1个元素a[n-1], 则for循环条件不满足, 循环直接结束, 执行a[i+1]=x, 即 a[n]=x;

输入: 20

输出: 4,7,9,15,18,20,22,26

4.1.4 一维数组的简单应用

【练习4.3】已知一个整型数组a[10]，其中已经存放有n个整数(n是小于10的)，但这n个数是无序的，现在输入一个整数x，再输入一个插入位置pos，把x插入到这个指定的位置，如果pos>n，则将x插入到最后一个元素的后面。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
14	7	29	5	18	32	26			

假设输入x=100, pos=3, 插入x后数组状态如下：

14	7	29	100	5	18	32	26		
----	---	----	-----	---	----	----	----	--	--

假设输入x=100, pos=9, 插入x后数组状态如下：

14	7	29	5	18	32	26	100		
----	---	----	---	----	----	----	-----	--	--

4.1.5 常见的排序方法

◆ **数据排序**：将数据按一定次序(从小到大/从大到小)进行排序

一般来说，需要进行排序的数据越少，操作越简单

- 1个整数排序，不需要任何操作
- 2个整数排序，需要进行1次比较，最多需要进行1次交换

例：对2个数进行从小到大排序

```
#include<stdio.h>
```

```
int main()
```

```
{ int a, b, t;
```

```
    scanf("%d%d", &a, &b);
```

```
    if(a>b)
```

```
        { t=a; a=b; b=t; }
```

```
    printf("%d, %d\n", a, b);
```

```
    return 0;
```

```
}
```

4.1.5 常见的排序方法

- 3个整数排序，需要进行3次比较，**需要0~3次交换**

例：对3个数进行从小到大排序

```
#include<stdio.h>
```

```
int main()
```

```
{ int a, b, c, t;
```

```
scanf("%d%d%d", &a, &b, &c);
```

```
if(a>b)
```

```
{ t=a; a=b; b=t; }
```

```
if(a>c)
```

```
{ t=a; a=c; c=t; }
```

```
if(b>c)
```

```
{ t=b; b=c; c=t; }
```

```
printf("%d,%d,%d\n", a, b, c);
```

```
return 0;
```

```
}
```

输入1: 1 2 3

3次比较，0次交换

输入2: 2 3 1

3次比较，2次交换

1 3 2

1 2 3

输入3: 3 2 1

3次比较，3次交换

2 3 1

1 3 2

1 2 3

4.1.5 常见的排序方法

- 4个整数排序，需要进行6次比较，需要0~6次交换

```
#include<stdio.h>
```

```
int main()
```

```
{ int a,b,c,d,t;
```

```
scanf("%d%d%d%d",&a,&b,&c,&d);
```

```
if(a>b) { t=a; a=b; b=t; }
```

```
if(a>c) { t=a; a=c; c=t; }
```

```
if(a>d) { t=a; a=d; d=t; }
```

3次比较，
排好第1个数a

```
if(b>c) { t=b; b=c; c=t; }
```

```
if(b>d) { t=b; b=d; d=t; }
```

2次比较，
排好第2个数b

```
if(c>d) { t=c; c=d; d=t; }
```

1次比较，
排好第3个数c

```
printf("%d,%d,%d,%d\n",a,b,c,d);
```

```
return 0;
```

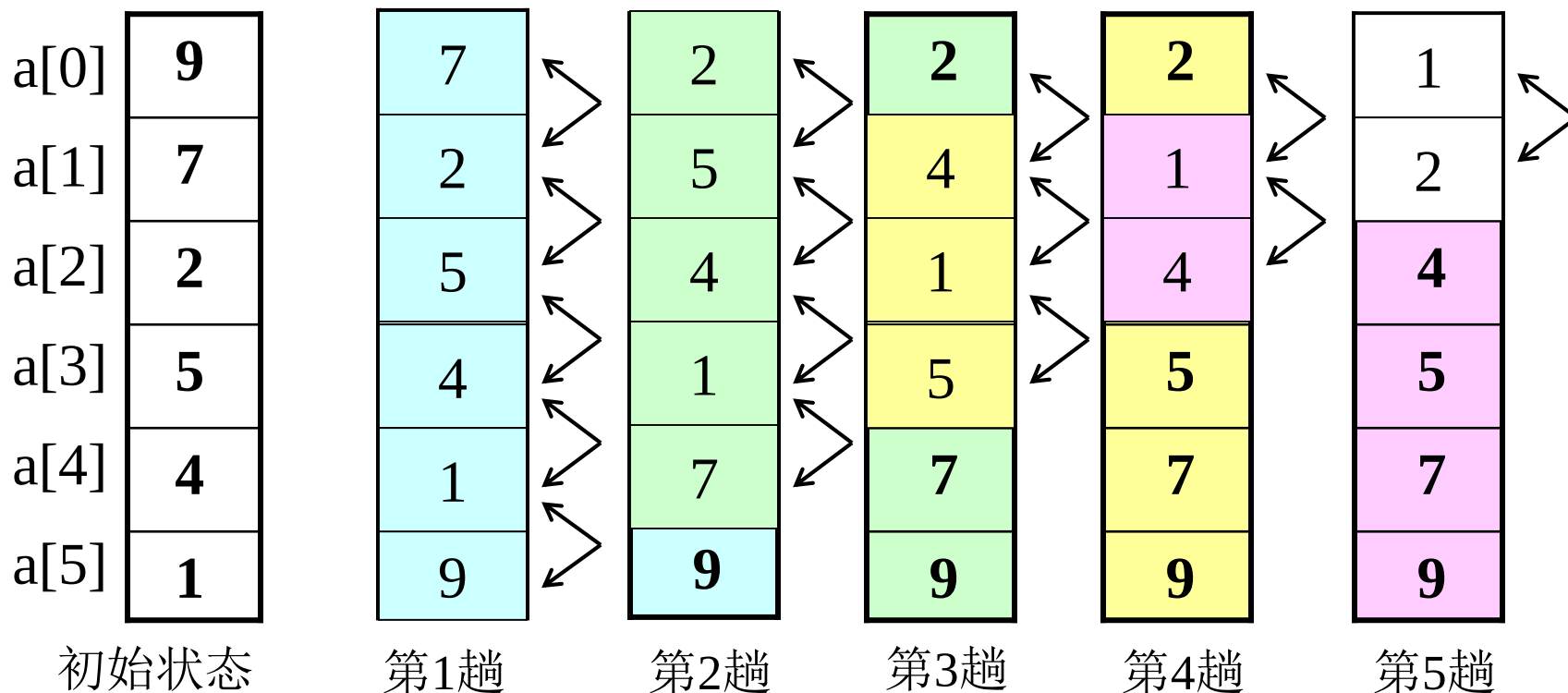
```
}
```

数据个数	比较次数
1	0
2	1
3	3
4	6
5	10
⋮	⋮

4.1.5 常见的排序方法

◆ 当待排序的数据越来越多时，就需要使用更有效的排序方法

➤ 用冒泡排序方法对6个整数进行从小到大排序



冒泡排序的基本思想：

依次比较相邻的两个数，将小数放前面，大数放后面。N个数排序需要进行N-1趟比较，从第1趟到第N-1趟，各趟的比较次数依次为：N-1次、N-2次 … 1次

4.1.5 常见的排序方法

➤ 冒泡排序代码

```
#include <stdio.h>
```

```
#define N 6
```

```
int main( )
```

```
{ int a[N], i, j, t;
```

```
  for ( i=0 ; i<N ; i++)
```

```
    scanf("%d", &a[i] );
```

```
    for ( i=0 ; i<N-1 ; i++)
```

```
      for ( j=0 ; j<N-1-i ; j++)
```

```
        if ( a[j]>a[j+1] )
```

```
          { t=a[j] ;
```

```
            a[j]=a[j+1] ;
```

```
            a[j+1]=t ;
```

```
          }
```

```
    for ( i=0 ; i<N ; i++)
```

```
      printf("%3d", a[i] );
```

```
    return 0;
```

```
}
```

用嵌套的for循环实现排序
外层循环控制进行几趟比较，
内层循环控制每一趟的比较次数

若前面的数大于后面的
数，则进行交换

4.1.5 常见的排序方法

➤ 思考：如何改进冒泡排序？

	初始状态	第1趟	第2趟	第3趟	第4趟	第5趟
a[0]	9	7	1	1	1	1
a[1]	7	1	2	2	2	2
a[2]	1	2	4	4	4	4
a[3]	2	4	5	5	5	5
a[4]	4	5	7	7	7	7
a[5]	5	9	9	9	9	9

因数组的初始数据可能是部分有序的，N个数排序不一定要全部完成 N-1趟的比较，当排序完成后，我们希望可以提前终止比较，应该如何实现？

4.1.5 常见的排序方法

➤ 用选择排序方法对6个整数进行从小到大排序

选择排序的基本思想：

- 先从N个数中找出最小的数, 把它与第一个位置的数交换
- 再从N-1个数中找出最小的数, 把它与第二个位置的数交换
- 这样重复做下去, 直到最后两个数进行比较为止

a[0]	1
a[1]	7
a[2]	2
a[3]	5
a[4]	4
a[5]	9

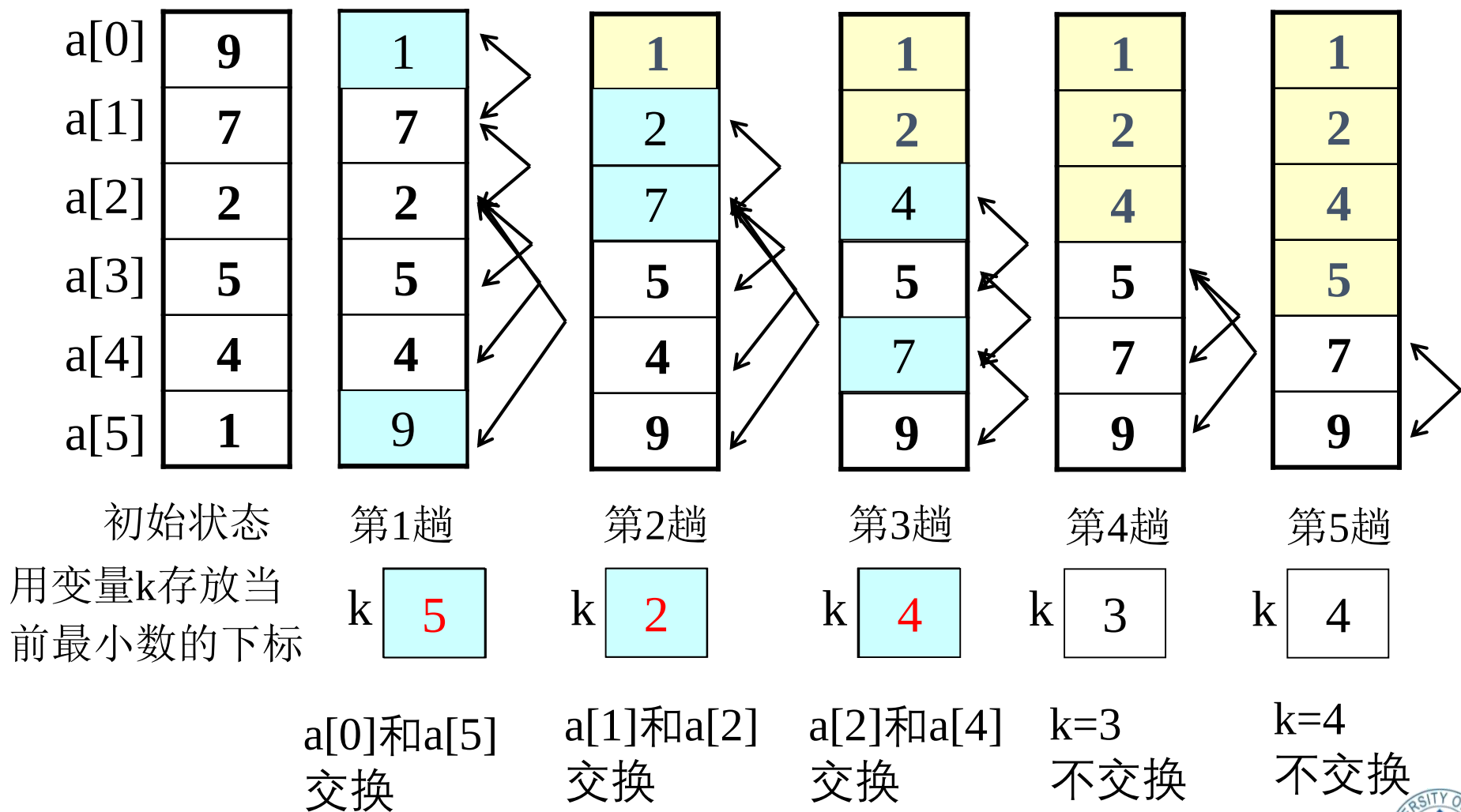
首先通过比较找到最小数1, 用变量k记录最小数的下标5, 而最小数应该存放在元素a[0]中, 所以最后将a[0]和a[k]进行交换

k

5

4.1.5 常见的排序方法

➤ 选择排序的过程演示



4.1.5 常见的排序方法

➤ 选择排序代码

```
#include <stdio.h>
```

```
#define N 6
```

```
int main( )
```

```
{ int a[N] , i , j , k , t;
```

```
  for ( i=0; i<N; i++)
```

```
    scanf("%d", &a[i] );
```

```
  for ( i=0; i<N-1; i++)
```

```
  { k=i;
```

```
    for ( j=i+1; j<N; j++)
```

```
    { if ( a[j]<a[k] ) k=j; }
```

```
    if ( k!=i )
```

```
    { t=a[i]; a[i]=a[k]; a[k]=t; }
```

```
  }
```

```
  for ( i=0; i<N; i++)
```

```
    printf("%3d", a[i] );
```

```
  return 0;
```

```
}
```

用嵌套的for循环实现排序
外层循环控制进行几趟比较

k存放当前最小数的下标
第i趟比较时, 其初值为i

若有比a[k]更小的数, 就修改k的值

若k的值不等于初值i, 说明a[k]
中是本趟找到的最小数, 应该用
a[k]与a[i]进行交换; 若k的值等
于初值i, 说明a[i]中就是本轮中
的最小数, 不需要交换

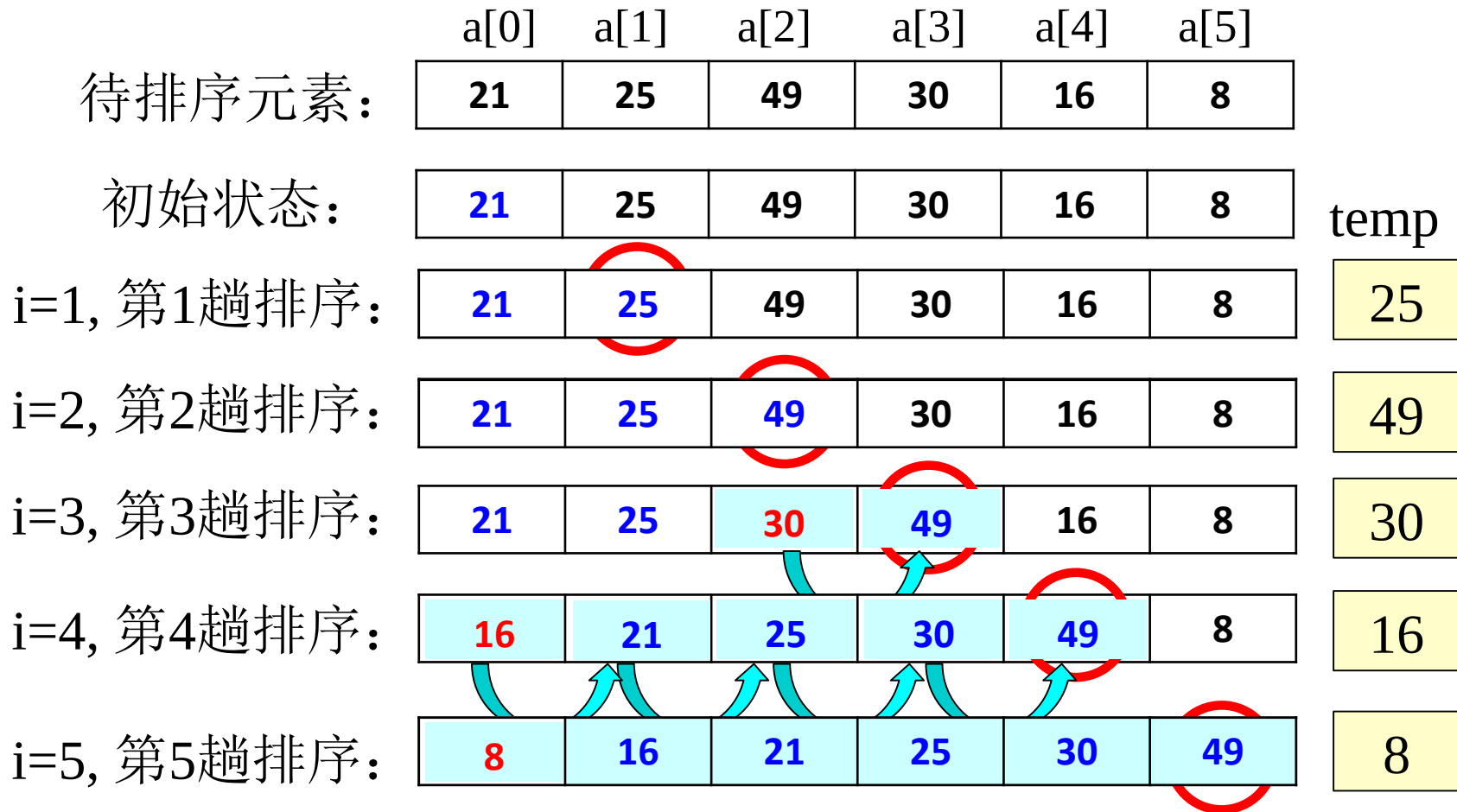
4.1.5 常见的排序方法

➤ 直接插入排序的基本思想

- 将待排序的序列分为有序序列和无序序列，依次从无序序列中取出元素值插入到有序序列的合适位置。
- 初始时，有序序列中只有第1个数，其余 $N-1$ 个数将组成无序序列，所以 N 个数排序需进 $N-1$ 趟操作。
- 在有序序列中寻找插入位置，可以从有序序列的最后一个数往前寻找，在未找到插入位置之前可以同时向后移动元素，为插入元素准备空间。

4.1.5 常见的排序方法

➤ 直接插入排序过程演示



4.1.5 常见的排序方法

➤ 直接插入排序代码

```
#include <stdio.h>
```

```
#define N 6
```

```
int main()
```

```
{ int a[N], i, j, temp;
```

```
for(i=0; i<N; i++) scanf("%d", &a[i]);
```

```
for(i=1; i<N; i++)
```

```
{ temp=a[i];
```

```
for( j=i-1; j>=0 && temp<a[j]; j-- )
```

```
{ a[j+1]=a[j]; }
```

```
a[j+1]=temp; //完成插入
```

```
}
```

```
for(i=0; i<N; i++)
```

```
printf("%3d", a[i]);
```

```
return 0;
```

```
}
```

外层循环控制进行几趟排序，N个数需要N-1趟

在有序序列(a[0]~a[i-1])中寻找插入位置，在寻找插入位置的同时完成元素移动

4.2 二维数组

4.2.1 二维数组的引出

【问题描述4.3】一个班有20名学生，每个学生参加了5门课程的考试，求每个学生5门课程的平均分。

	英语	高数	物理	政治	C语言
学生1	70	80	85	75	90
学生2					
学生3					
⋮					
⋮					
⋮					
⋮					
学生20					

每个学生有一行数据，
是一个一维数组

20个学生有20行数据，
形成一个二维的表格，
这个二维表格其实就
是一个二维数组

4.2.2 二维数组的定义与引用

1. 二维数组的概念: 一个一维数组, 它的每一个元素都是类型相同的一维数组, 就形成一个二维数组
2. 二维数组的定义形式

类型标识符 数组名 [常量表达式1] [常量表达式2]

例: `int a[3][4];`

	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
<code>a[0]</code>				
<code>a[1]</code>				
<code>a[2]</code>				

数组名后的第一个方括号表示二维数组的第一维, 第二个方括号表示第二维, 总的数组元素个数为两维长度的乘积。

4.2.2 二维数组的定义与引用

3. 二维数组元素的引用形式: **数组名[下标1][下标2]**

➤ 说明

(1) 每个下标都要用 `[]` 括起来, 如 `a[2][1]` 不能写成 `a[2,1]`

(2) 下标可以是整型表达式或整型常量, 其值**从0开始**,
不能超过数组定义的范围下标不要超过定义的范围

【问题描述4.3】20名学生的五门课程的成绩可定义如下数组:

float score[20][5];

定义了一个二维的实型数组, 该数组含有 20×5 个数组元素

score[0][0]	score[0][1]	score[0][2]	score[0][3]	score[0][4]
score[1][0]	score[1][1]	score[1][2]	score[1][3]	score[1][4]
⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮
score[19][0]	score[19][1]	score[19][2]	score[19][3]	score[19][4]

4.2.2 二维数组的定义与引用

➤ 【问题描述4.3】 参考程序

```
1. #include <stdio.h>
2. #define M 20    //定义符号常量M，代表学生总人数
3. #define N 5     //定义符号常量N，代表5门课程
4. int main()
5. {
6.     int i, j;    double fScore[M][N], fAver[M];
7.     printf("\nInput %d students' %d scores(fScore[%d][%d]):\n", M, N, M, N);
8.     for(i=0; i<M; i++)    //外层循环，控制学生人数
9.     { fAver[i]=0;          //第i个学生的总成绩初值为0
10.        for(j=0; j<N; j++) //内层循环，控制课程
11.        { scanf("%lf", &fScore[i][j]);    //输入第i个学生的成绩
12.            fAver[i]=fAver[i]+fScore[i][j]; //计算第i个学生的总成绩
13.        }
14.        fAver[i]=fAver[i]/N;
15.    }
```

每个学生的平均成绩
存放在一维数组中

4.2.2 二维数组的定义与引用

➤ 【问题描述4.3】 参考程序

```
16.  printf("\nScore1  Score2  Score3  Score4  Score5  average:\n");
17.  for(i=0; i<M; i++)
18.  { for(j=0; j<N; j++)           //输出第i学生的5门课程的成绩
19.      printf("%-8.2lf", fScore[i][j]);
20.      printf("%-8.2lf\n", fAver[i]);   //输出第i个学生的平均成绩
21.  }
22.  return 0;
23. }
```

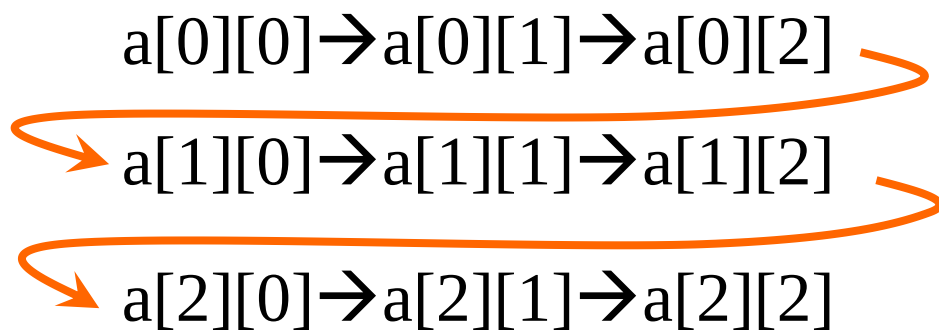
4.2.2 二维数组的定义与引用

4. 二维数组的存储形式

在内存中，二维数组的元素是连续存放的，C语言中的存放顺序是按行存储，即先顺序存放第一行的元素，再存放第二行元素。

例如： `int a[3][3];`

数组a的存放形式如下：



注意：二维数组的数组名表示数组在内存中的首地址，是一个地址常量。

0x0012ff64		a[0][0]
0x0012ff68		a[0][1]
0x0012ff6c		a[0][2]
0x0012ff70		a[1][0]
0x0012ff74		a[1][1]
0x0012ff78		a[1][2]
0x0012ff7c		a[2][0]
0x0012ff80		a[2][1]
0x0012ff84		a[2][2]

4.2.3 二维数组的初始化

1. 分行初始化 (强烈推荐)

```
int a[3][4]={ { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 } };
```

第1对{ }内的数据赋给第1行数组元素,
第2对{ }内的数据赋给第2行数组元素,
以此类推, 此方法比较直观, 不易出错

1	2	3	4
5	6	7	8
9	10	11	12

2. 按数据的排列顺序对数组元素赋初值

```
int a[3][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

将数据依次赋给元素 a[0][0] , a[0][1] ...a[2][3]

注意: 此方法数据没有明显的界限, 当数据较多时容易出错

4.2.3 二维数组的初始化

3. 对数组的部分元素赋初值

```
int a[3][4]={1, 2},{3},{0, 4};
```

1	2	0	0
3	0	0	0
0	4	0	0

```
int a[3][4]={1,2,3,4,5,6};
```

1	2	3	4
5	6	0	0
0	0	0	0

4. 对数组全部元素赋初值时可省略第一维的长度，系统根据数据的个数和第二维的长度自动求出第一维的长度

```
int a[ ][4]={ {1, 2}, { 0, 3, 4 }, { 5 } };    数组a第一维长度为 3
```

```
int b[ ][2]={ 1, 2, 3, 4, 5, 6, 7, 8 };        数组b第一维长度为 4
```

5. 按行对部分元素赋初值，可以省略第一维的长度说明

```
int a[ ][3]={0,0,3}, {0,2}, {1};
```

4.2.4 二维数组的应用

【例4.7】 找出矩阵中最大的数,并输出其行号和列号

```
#include <stdio.h>
```

```
int main( )
```

```
{ int i , j, max, row, colum;
```

```
int a[3][4]={ {5,2,0,9},{3,7,12,6},{10,4,1,8}};
```

```
max=a[0][0]; row=0; colum=0;
```

```
for ( i=0; i<3; i++ )
```

```
    for ( j=0; j<4; j++ )
```

```
        if ( a[i][j]>max )
```

```
            { max=a[i][j] ;
```

```
              row=i ;
```

```
              colum=j ;
```

```
            }
```

```
printf("max=%d, row=%d, colum=%d\n", max, row, colum);
```

```
return 0;
```

```
}
```

5	2	0	9
3	7	12	6
10	4	1	8

max

12

row

1

colum

2

输出: max=12, row=1, colum=2

4.2.4 二维数组的应用

【例4.8】 将一个矩阵进行转置(即原来的行变为列)

```
#include<stdio.h>
```

```
int main( )
```

```
{ int a[3][4], b[4][3], i, j;
```

```
for ( i=0 ; i<3 ; i++ )
```

```
for ( j=0 ; j<4 ; j++ )
```

```
scanf("%d", &a[i][j] );
```

输入数组a

5	2	0	9
3	7	12	6
10	4	1	8

a[0][2]

```
for ( i=0 ; i<3 ; i++ )
```

```
for (j=0 ; j<4 ; j++)
```

```
b[j][i]=a[i][j];
```

进行矩阵转置

```
for ( i=0 ; i<4 ; i++ )
```

```
{ for ( j=0 ; j<3 ; j++ )
```

```
{ printf("%5d", b[i][j]); }
```

```
printf("\n");
```

输出数组b,
注意内外层循
环条件的改变

5	3	10
2	7	4
0	12	1
9	6	8

a[2][1]

b[2][0]

b[1][2]

```
}
```

```
return 0;
```

```
}
```

这个输出换行是
外层循环的语句

4.3 字符数组

4.3.1 字符数组的引出

在实际问题中，经常会遇到使用字符的情况，比如姓名、书名、英文单词、英文句子等等。这些都是由多个字符组成的，当把多个字符进行整体考虑，就涉及到字符串的概念。C语言中有字符类型，但并没有字符串类型，所以C语言中字符串是用字符数组存放的。

【问题描述4.4】生活中使用密码是常事。用户预先设置一个N位长度的密码，密码由数字、字母、符号等组成。当程序运行时要求用户输入密码，3次之内，按用户输入的密码正确与否，输出“欢迎！”、“密码错误！”；超过3次后输出“退出！”。

分析：

首先要预设一个长度为N的密码，一个密码由多个字符组成，因此需要使用字符数组来存放密码。

4.3.2 字符数组的定义与引用

1. 字符数组的概念

用来存放字符型数据的数组称为字符数组，它主要用于存储一串连续的字符。每1个数组元素只能存放1个字符。

2. 字符数组的定义格式

char 数组名[整型常量表达式]

例：char c[10];

问题描述4.4的解决方法：

用一个长度为N的字符数组code[N]来存放预设的密码。

再定义一个字符数组user[N]，用来存放用户输入的密码。

只要将预设密码code[N]与用户输入密码user[N]里面的每一个字符依次进行比较，所有字符都相等时表示密码正确，只要比较过程中有一个字符不相等，就说明密码错误。

4.3.2 字符数组的定义与引用

3. 字符数组的初始化

- (1) 为数组中的元素指定初值, 如果初值个数小于数组长度, 按顺序赋值后, 其余元素自动赋空字符'\0'。

例: `char c[8] = { 'H', 'e', 'l', 'l', 'o' };`

H	e	l	l	o	\0	\0	\0
---	---	---	---	---	----	----	----

- (2) 如果对数组全部元素指定初值, 则可省略数组的长度

例: `char c[] = { 'H', 'e', 'l', 'l', 'o' };`

H	e	l	l	o
---	---	---	---	---

- (3) 二维字符数组初始化

例: `char c[2][3] = { { 'a', 'b', 'c' }, { 'A', 'B', 'C' } };`

a	b	c
A	B	C

4.3.3 字符串

1. 字符串的定义

由若干个有效字符组成的序列，有效字符包括字母, 数字, 专用字符, 转义字符。

- 字符串常量是由双引号括起来的字符序列。

如: "bfer" "a45-7" "m\tk\n"

- 字符串的存储

ANSI C语言没有提供字符串类型，因此没有字符串变量，字符串的处理都是依靠字符数组来完成。

字符串必须以'\0'作为结束标志，它只是表示字符串的结束，没有任何的具体含义。'\0'不会计入字符串的长度，但'\0'会占用一个字节的存储空间，因此定义字符数组时，要保证数组长度始终大于字符串的实际长度。

4.3.3 字符串

例 : `char c[5];`

① `c[0]='O' ; c[1]='K' ; c[2]='!' ;`

② `c[0]='O' ; c[1]='K' ; c[2]='!' ; c[3]='\0' ;`

O	K	!		
---	---	---	--	--

O	K	!	\0	
---	---	---	----	--

注意：字符数组与字符串并不相同

2. 字符串的初始化

`char c[ ] = { "Hello" };`

`char c[6] = { "Hello" };`

`char c[ ] = "Hello" ;`

`char c[6] = "Hello" ;`

`char c[6] = { 'H','e','l','l','o' };`

`char c[6] = { 'H','e','l','l','o','\0' };`

推荐使用

等价形式

4.3.3 字符串

➤ 提问：以下初始化正确吗？

`char c[ ] = "Hello";`

H	e	l	l	o	\0
---	---	---	---	---	----

 正确

等价于 `char c[6] = "Hello";`

`char c[5] = "Hello";`

H	e	l	l	o	\0
---	---	---	---	---	----

 错误

注意：初始化时数组长度应足够大，确保容纳所有字符和'\0'

`char c[5] = { 'H','e','l','l','o' };`

H	e	l	l	o
---	---	---	---	---

 正确

注意：这不是字符串初始化，而是字符数组初始化
字符数组并不要求必须包含'\0'

4.3.3 字符串

3. 字符串的输入输出

(1) 用"%c"格式控制符或getchar()/putchar()实现单个字符的输入输出

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
    int i=0;
```

```
    char str[20];
```

```
    while ( (str[i]=getchar( )) != '\n') //输入字符不是回车符时执行循环
```

```
        {      i++;      }
```

```
    str[i]='\0';      //在最后输入的字符后面加'\0'字符串结束标志
```

```
    for (i=0; str[i] != '\0'; i++) //字符不是'\0'时执行循环
```

```
        printf("%c", str[i] ); //输出单个字符
```

```
    return 0;
```

```
}
```

4.3.3 字符串

3. 字符串的输入输出

(2)用"%s"格式控制符对字符串进行整体输入和输出。

```
#include <stdio.h>
int main( )
{
    char str[20];
    scanf(" %s", str);
    printf(" %s\n", str);
    return 0;
}
```

注意：

- ① 从键盘输入字符串时不需加双引号。
- ② 用scanf输入字符串时，空格和回车符都会作为字符串的分隔符，**即scanf的"%s"格式不能用来输入包含有空格的字符串。**
- ③ 用scanf和printf的"%s"格式输入输出字符串时，**输入项、输出项都是字符数组名**，而不是数组元素。
- ④ 如果数组长度大于字符个数时，只输出到'\0'结束标志符为止。

例如：

```
char c[10]= {'A', 'B', 'C', '\0', 'D', 'E'};
printf(" %s", c);    //只输出"ABC"
```

4.3.3 字符串

3. 字符串的输入输出

(3)用gets()函数和puts()函数实现字符串的输入输出。

说明:

- ① **gets**函数可以输入包括空格在内的全部字符，直到遇到回车符为止。
- ② **gets**输入字符串时，若输入字符数大于字符数组的长度，则多出的字符会存放在数组的合法存储空间之外，可能会影响其他数据。

例: `int i;`

`char c[5];`

设输入为: abc def✓

① `for ( i=0 ; i<5 ; i++ )`
`scanf("%c", &c[i]);`

a	b	c		d
---	---	---	--	---

② `scanf("%s", c);`

a	b	c	\0	
---	---	---	----	--

③ `gets ( c );`

a	b	c		d	e	f	\0
---	---	---	--	---	---	---	----

存在隐患

4.3.3 字符串

(3)用gets()函数和puts()函数实现字符串的输入输出。

说明:

- ③ puts函数一次输出一个字符串，输出字符不包括结束符'\0'，
输出时将'\0'自动转换成换行符
- ④ 如果字符数组中包含多个'\0'，则遇到第1个'\0'时就结束输出。

例: `char s1[5]="abc", s2[10]="defg";`
`printf( "%s%s\n", s1, s2 );`
`puts( s1 );`
`puts( s2 );`

输出:
abcdefg
abc
defg

例: `char str[10]={'a', 'b', 'c', '\0', 'd', 'e', '\0'};`
`puts(str);`

输出: abc

a	b	c	\0	d	e	\0	\0	\0	\0
---	---	---	----	---	---	----	----	----	----

4.3.3 字符串

4. 字符串的处理函数

- 由于字符串的特殊性，很多常规操作都不能用处理数值型数据的方法来完成，例如：赋值、比较操作等。
- 此外，字符串还有一些特殊的操作，例如：计算字符串的长度、字符串的连接等。
- ANSI C语言中为字符串定义了一系列字符串处理函数来完成这些工作，这些函数都包含在头文件“string.h”中，在使用这些函数时需要在程序的开头包含该头文件 **#include <string.h>**

char s1[5]="abc" , s2[5] , s3[8] ;

s2 = "abc"; // 错，不能这样赋值，只有初始化时这样写才对

s3 = s1; // 错，不能将s1 赋值给s3

➤ 提问：这种赋值错误的本质原因是什么？

数组名是一个地址常量，不可能给一个常量进行赋值

4.3.3 字符串

(1) 字符串复制函数strcpy

函数原型: **char *strcpy(char *s1, const char *s2);**

调用格式: **strcpy(字符数组1, 字符串2)**

功能: 将字符串2复制到字符数组1中。

char s1[5]="abcd" , s2[8] , s3[3] ;

a	b	c	d	\0
---	---	---	---	----

strcpy (s2 , s1) ;

a	b	c	d	\0	随	机	值
---	---	---	---	----	---	---	---

strcpy (s3 , "good") ;

g	o	o	d	\0
---	---	---	---	----

存在隐患

说明:

- ① 字符数组1的长度不应小于字符串2的长度, 复制时连同 '\0' 一起复制
- ② 字符数组1必须是**数组名或字符型指针变量**。

字符串2可以是**数组名、字符串常量, 或字符型指针变量**。

4.3.3 字符串

(2) 有限制的字符串复制函数**strncpy**

函数原型: `char *strncpy(char *s1, const char *s2, unsigned int n);`

调用格式: `strncpy(字符数组1, 字符串2, n)`

功能: 将字符串2的前n个字符复制到字符数组1中，如遇到'\0'则提前结束复制。

例: `char c1[10]="program", c2[ ]="abcd";`
`strncpy( c1 , c2 , 3 );`

注意这里还是原来的字符

c1	a	b	c	g	r	a	m	\0	\0	\0
----	---	---	---	---	---	---	---	----	----	----

c2	a	b	c	d	\0
----	---	---	---	---	----

➤ **提问:** `strncpy( c1 , c2 , 6 );`
复制后c1的内容可能是什么?

4.3.3 字符串

【例4.15】 C语言期末考试后，老师想知道班里最高分的学生姓名，请编程实现。首先输入学生的人数 n ($n \leq 100$)，然后再输入每位学生的分数和姓名，分数 ≤ 100 ，是一个非负整数，姓名为没有空格的字符串，长度不超过20。输出最高分数的学生姓名，输入数据保证最高分只有一位同学。

【输入】：

4
89 Mary
92 John
96 Alex
87 Mike

【输出】：

Alex

分析：

这个题实际是找最大值，不过每个数值都是与一个姓名（即字符串）相关，寻找最大值的同时要记录对应的姓名。

4.3.3 字符串

【例4.15】 参考程序

```
#include <stdio.h>
#include <string.h>
int main ()
{   int i, n, score, max=0;
    char name[21], max_name[21];
    scanf("%d", &n);
    for(i=1; i<=n; i++)
    {   scanf("%d%s", &score, name);
        if( score>max )
        {   max=score;
            strcpy(max_name, name);
        }
    }
    printf("%s\n", max_name);
    return 0;
}
```

这个数组用来保存
最高分的姓名

输入成绩和姓名

如果当前输入的成绩大于
最高分，则给max重新赋值，
同时将姓名拷贝到
max_name数组中

4.3.3 字符串

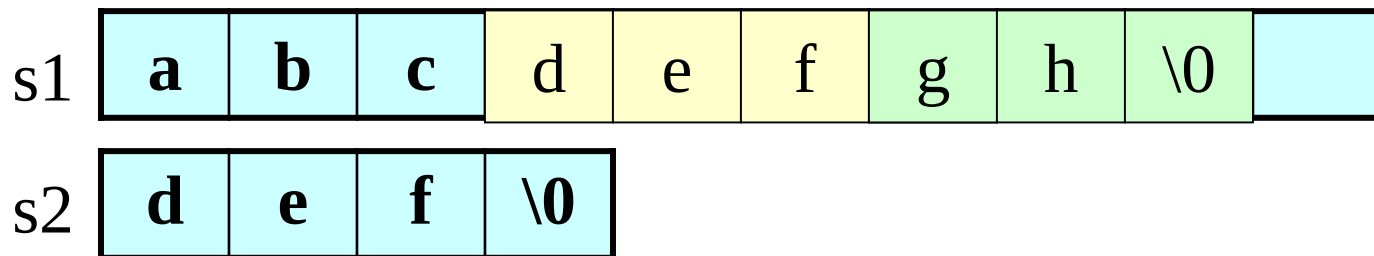
(3) 字符串连接函数strcat

函数原型: `char *strcat(char *s1, const char *s2);`

调用格式: `strcat(字符数组1, 字符串2)`

功能: 把字符串2连接到字符串1的后面, 连接后的字符串仍存放在字符数组1中。

例: `char s1[10]="abc" , s2[ ]="def" ;`
`strcat ( s1, s2 );     strcat ( s1, "gh" );`



说明:

- ① 字符数组1必须定义得足够大, 以便容纳连接后的新字符串。
- ② 连接前两个字符串后面都有一个 '\0', 连接时将字符串1后的 '\0' 去掉, 只在新字符串的最后保留一个 '\0'。

4.3.3 字符串

(4) 有限制的字符串连接函数**strncat**

函数原型: `char *strncat(char *s1, const char *s2, unsigned int n);`

调用格式: `strncat(字符数组1, 字符串2, n)`

功能: 把字符串2中的n个字符连接到字符数组1的字符串后边，如遇到 '\0' 则提前结束。

例如:

`char str1[15]="China ", str2[ ]="Bei jing";`

`strncat(str1, str2, 3);`

`puts(str1);`

str1:

C	h	i	n	a		B	e	i	\0	\0	\0	\0	\0	\0
---	---	---	---	---	--	---	---	---	----	----	----	----	----	----

str2:

B	e	i		j	i	n	g	\0
---	---	---	--	---	---	---	---	----

4.3.3 字符串

【例】自动生成密码。

输入名字和生日，名字为20个字符以内的字符串，中间无空格，生日为8个字符的字符串，其中年占4个字符，月和日均占2个字符，如：19970701。密码生成规则是名字的前2个字符加生日的8个字符。

【输入】

ZhangYang

19970701

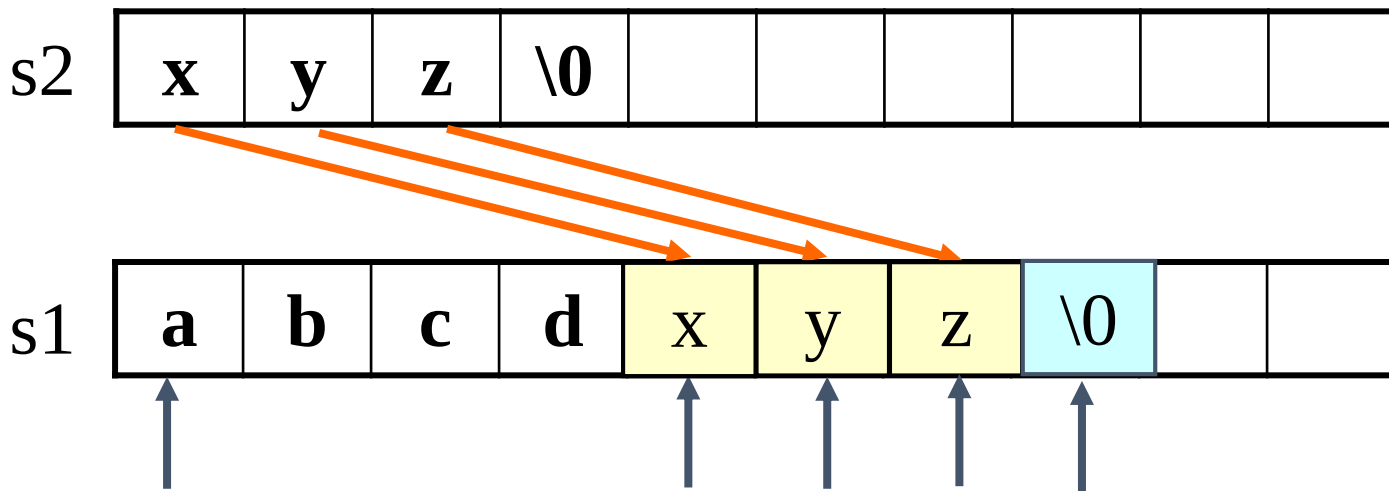
【输出】

Zh19970701

```
#include<stdio.h>
#include<string.h>
int main()
{
    char code[16]=""; //空串
    char name[21], birthday[10];
    gets(name);
    gets(birthday);
    strncat(code, name, 2);
    strcat(code, birthday);
    puts(code);
    return 0;
}
```

4.3.3 字符串

【例4.9】 编程实现两个字符串的连接,但不能使用`strcat`函数



➤ 目标: 将字符串s2 连接到字符串s1后面

步骤1: 让数组s1 的下标指向字符串的末尾, 即'\0'的位置

步骤2: 依次将数组s2 的字符赋给s1, 它们的下标都加1

步骤3: 最后数组s1 的末尾赋值为'\0'

4.3.3 字符串

【例4.9】参考程序1

```
#include <stdio.h>
```

```
int main()
```

```
{  char s1[80], s2[40];  int i=0, j=0;
```

```
    gets(s1);    gets(s2);
```

```
    while(s1[i]!='\0')
```

```
    {  i++;  }    //步骤1: 让s1的下标指向字符串末尾
```

```
    while(s2[j]!='\0')
```

```
    {  s1[i]=s2[j];  //步骤2: 依次将数组s2的字符赋给s1
```

```
        i++;  j++;  //数组s1和s2的下标都加1
```

```
    }
```

```
    s1[i]='\0';    //步骤3: 数组s1的最后加字符串结束标志
```

```
    printf("The new string is: %s\n", s1);    //用%s输出字符串
```

```
    return 0;
```

```
}
```

4.3.3 字符串

(5) 字符串比较函数strcmp

函数原型: `int strcmp(const char *s1, const char *s2);`

调用格式: `strcmp(字符串1, 字符串2)`

功能: 两个字符串可以是字符串常量或字符数组变量。

比较时从左到右逐个字符比较其ASCII值的大小，直到出现不一样的字符或遇到'\0'为止，比较结果由函数返回。

- ① 字符串1 == 字符串2，函数的返回值为0。
- ② 字符串1 > 字符串2，函数的返回值为正整数。
- ③ 字符串1 < 字符串2，函数的返回值为负整数。

例: `int n1, n2, n3;`

`char s1[5] = "def", s2[5] = "dab";`

`n1 = strcmp("abc", "def");` //字符'a'与'd'比较, n1=-1

`n2 = strcmp(s1, "def");` //所有字符都相等, n2=0

`n3 = strcmp(s1, s2);` //字符'e'与'a'比较, n3= 1

4.3.3 字符串

(6) 测试字符串长度函数strlen

函数原型: **unsigned int strlen(const char *s);**

调用格式: **strlen(字符串)**

功能: 测试字符串的长度, 函数值为字符串中字符的个数, 不包括'\0'在内。

例如:

```
char str[20]="China";
```

```
printf("%d\n", strlen(str)); //输出为5
```

```
int n;
```

```
n=strlen("welcome"); //n的值为7
```

4.3.3 字符串

【例4.9】参考程序2（两个字符串的连接不用strcat函数）

```
#include <stdio.h>
```

```
int main()
```

```
{   char s1[80], s2[40];   int i=0, j=0;
```

```
    gets(s1);    gets(s2);
```

```
    i=strlen(s1);
```

//步骤1:让s1的下标指向字符串末尾

```
    while(s2[j]!='\0')
```

```
    {
```

```
        s1[i++]=s2[j++];
```

//步骤2: 依次将数组s2的字符赋给s1
//数组s1和s2的下标都加1

```
    }
```

```
    s1[i]='\0';
```

//步骤3:数组s1的最后加字符串结束标志

```
    puts(s1);
```

//用puts()输出字符串

```
    return 0;
```

```
}
```

4.3.3 字符串

【问题描述4.4】用字符数组实现的程序代码

```
#include <stdio.h>
```

```
#define N 9 //N表示密码的长度
```

```
int main()
```

```
{ char code[N]={ 'a', 'b', 'c', '1', '2', '*', 'X', 'Y', 'Z'}; //预设的密码
```

```
char user[N]; int i, k, t=0;
```

```
while(1)
```

```
{ k=1; //定义预设密码与用户输入密码位是否相等的标志k
```

```
printf("Please input your key:");
```

```
for(i=0; i<N; i++) user[i]=getchar(); //输入密码
```

```
for(i=0; i<N; i++) //利用循环比较输入密码与原密码的每一个字符
```

```
if(code[i]!=user[i])
```

```
{ k=0; break; } //任一字符不相同时令k=0, 结束for循环
```

```
t++; //尝试次数t加1
```

```
if(k==0) printf("密码错误!\n");
```

```
else { printf("欢迎!\n"); break; }
```

```
if(t>=3) { printf("退出!\n"); break; }
```

```
} //end while(1)
```

```
return 0;
```

```
}
```

现在的任务是将
这个程序改造成
用**字符串**来实现

4.3.3 字符串

【问题描述4.4】用字符串实现的程序代码

```
#include <stdio.h>
```

```
#define N 20           //密码长度
```

```
int main()
```

```
{ char code[N]="abc12*XYZ"; //字符串初始化, 预设密码
```

```
  char user[N];   int i, k, t=0;
```

```
  while(1)
```

```
  { printf("Please input your key:");
```

```
    gets(user);           //输入密码
```

```
    k=strcmp(code, user); //用strcmp函数比较密码
```

```
    t++;
```

```
    if(k==0) { printf("欢迎!\n"); break; } //k为0表示密码正确
```

```
    else printf("密码错误!\n");
```

```
    if(t>=3) { printf("退出!\n"); break; }
```

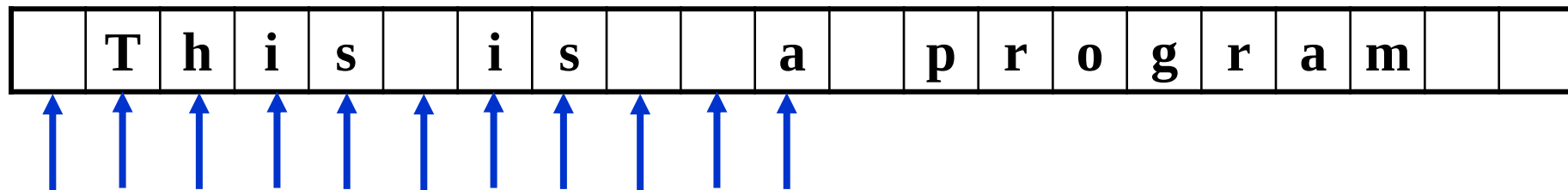
```
  } //end while(1)
```

```
  return 0;
```

```
}
```

4.3.3 字符串

【例4.11】 输入一行字符, 统计其中有多少个单词



word 1 num 3

分析:

单词的数目可以由空格出现的**次数**决定。如果测出某一个字符为非空格, 而它前面的字符是空格, 则表示“新的单词开始了”, 此时单词计数器 num 加1。如果当前字符为非空格, 而其前面的字符也是非空格, 则意味着仍然是原来那个单词的继续, num 的值不变。

实现方法:

设置一个标志变量 word, 如果当前字符是空格, 则 word=0;
如果当前字符不是空格, 则 word=1; 当 word 由 0 变为 1 时,
计数器 num 的值加 1

4.3.3 字符串

【例4.11】 参考程序

```
#include <stdio.h>
```

```
int main( )
```

```
{ char c, string[81];
```

```
int i, num=0, word=0;
```

```
gets(string);
```

```
for(i=0; (c=string[i])!= '\0'; i++)
```

```
if (c==' ') word=0;
```

```
else
```

```
if (word==0)
```

```
{ word=1;
```

```
num++; }
```

```
printf ("There are %d words.\n", num);
```

```
return 0;
```

```
}
```

num用来统计单词个数, **word**判别字符是否为空格, **word=0**表示字符是空格, **word=1**表示字符不是空格

c不是空格, 若**c**前面的字符是空格, 表明这是一个新单词的开始, 则**word**由0改为1, 计数器**num**加1; 若**c**前面的字符不是空格, 则不作处理



培养创新思维 提升编程能力